



Chatsuite Platform

Estado del proyecto

Fecha	18 de agosto de 2026
Repositorio	github.com/DTGrowthPartners/chatsuite-platform
Panel	dtgp.ai
Servidor	VPS DT Growth Partners

Chatsuite Platform — estado del proyecto

Actualizado: 2026-08-18 **Repo:** github.com/DTGrowthPartners/chatsuite-platform **Panel:** <https://dtgp.ai>

En una línea

Dar de alta un cliente completo —su Chatwoot con marca propia, su bot de IA y su WhatsApp— es un formulario de tres minutos. Antes eran diez pasos manuales, una imagen Docker de 4 GB por cliente y un bot armado a mano.

Qué está listo

	Pieza	Estado
1	Imagen única <code>chatsuite:base</code> , marca en runtime	✓ sin build por cliente
2	Provisioner de 8 pasos, idempotente	✓ probado
3	Panel en <code>dtgp.ai</code> con login propio	✓ en producción
4	Motor de bots (<code>bot-engine/</code>)	✓ un código, un perfil por cliente
5	Configurador del bot con simulador	✓ 10 pestañas
6	Canal de WhatsApp: Evolution + QR	✓ un Evolution por cliente
7	Módulo <code>tienda</code>	✓ catálogo, pedidos, domicilios, PDF
8	Módulo <code>citas</code>	✓ agenda propia, servicios, cancelaciones
9	Medición interna	✓ eventos, contención, tokens
10	Suspender / reanudar servicio	✓ sin borrar nada

Capacidad: ~8 clientes completos por nodo (1,4 GB cada uno: Chatsuite 1,1 GB + Evolution 190 MB + bot 80 MB).

Cómo se da de alta un cliente

1. **Panel** → **+ Nuevo cliente**. Nombre, subdominio, correo y logo. En *Más opciones*: marca comercial, sitio, idioma, zona horaria, ciudad y si se crea el bot de una vez. Tarda ~3 minutos.

2. **Tarjeta** → **Bot: configurar y probar**. Se configura por pestañas y se afina en el **simulador**, que corre el bot de verdad sin mandar nada a WhatsApp. Arranca en borrador: no le escribe a nadie.
3. **Tarjeta** → **WhatsApp: conectar el número**. Levanta su Evolution, muestra el QR y, al conectar, **Enlazar el bot al inbox**.
4. **Pasar el bot a producción** cuando esté afinado.

Los módulos

Un módulo agrupa tres cosas que van juntas: sus herramientas, sus handlers y su parte del prompt. Un cliente de citas no ve una palabra sobre domicilios.

Módulo	Qué hace	Estado
comun	escalar a humano, avisar al equipo	siempre activo
tienda	catálogo con fotos, pedidos, domicilios, catálogo en PDF	✓
citas	disponibilidad real, agendar, consultar, cancelar	✓
socios	padrón, planes, vencimientos (ACBfit)	☐ falta decidir de dónde sale el padrón
tareas	tablero interno contra DTOS (Nanoplush)	☐ el más fácil: DTOS ya tiene API

`tienda` es genérico: los atributos salen del propio catálogo, así que sirve para computadores igual que para ropa.

`citas` trae agenda propia (disponibilidad en el perfil, citas en `citas.json`), con proveedor intercambiable para conectar Cal.com o AgendaPro sin reescribirlo.

Qué ve cada quien

El cliente solo recibe su plataforma con su marca y su WhatsApp: mira cómo contesta el bot e interviene cuando quiere. No hay panel aparte — los pedidos y las citas salen en la barra lateral de la conversación y en bandejas filtradas.

Nosotros vemos las métricas en el panel: contención, mensajes atendidos, pedidos, tokens, fallos, y la lista de lo que el bot no supo responder. Lo financiero vive en DTOS.

Lo que falta

Por decidir contigo

- **ACBfit**: dónde vive el padrón hoy (hoja de cálculo, sistema con API, o nada). Si no hay nada, se hace igual que el catálogo y el módulo queda siendo su sistema.

- **Nanoplush:** confirmar que el bot administrativo va contra las tareas de DTOS.

Trabajo pendiente

- Migrar Tu Bodega al motor. **El perfil ya está armado y probado**, corriendo en borrador (`bot - tubodega - ensayo` , puerto 3311). El cambio del fin de semana es apuntar el AgentBot y encenderlo. ⚠ `tubodega - bot` se sigue editando: hay que portar lo que aterrice durante la semana.
- Cloud API desde el panel: el motor lo soporta, pero los tokens de Meta se escriben a mano en el `.env` . Falta la pantalla.
- Monitor de salud de los bots del motor.
- Recordatorios de cita: son tráfico proactivo y con Cloud API exigen plantilla aprobada. Con Evolution no hay problema.

Menores

- `acbfit` , `cantina` , `equilibrio` y `tubodega` tienen registro A en dtgp.ai pero **no están en la lista de slugs reservados**: un alta podría pisarlos.
- El sudoers acotado sigue sin instalar.
- ~10,5 GB en imágenes Docker viejas sin usar.
- 13 registros de Tu Bodega con `precio: 0` que alguien debería revisar: ahí el 0 significa «no sé», no «gratis».

Lo que costó encontrar

Ninguna de estas avisa cuando falla. Están todas resueltas en el código.

Del canal y Chatwoot

- `enable_auto_assignment` en `true` hace que la conversación nazca asignada y **el bot no la vea nunca**. Todo parece bien y simplemente no contesta.
- Hacen falta **dos tokens**: el del Agent Bot escribe mensajes, pero leer el historial y poner etiquetas exige el de un usuario agente. Con uno solo da 401 en silencio.
- `POST /labels` **reemplaza** la lista entera, no agrega.
- Chatwoot **crea al vuelo** cualquier etiqueta que reciba: sin catálogo cerrado se llena de etiquetas inventadas.
- Sin la **definición** del atributo, Chatwoot guarda el valor pero no lo muestra ni deja filtrar por él.
- Escribir atributos **re-dispara** `conversation_updated` : hay que leer antes y escribir solo si algo cambió.
- Con Cloud API (webhook v26) **el contacto no trae teléfono**: exigirlo descarta el 100% del tráfico sin un error en el log.
- Crear el **DNS antes** de levantar Evolution, o el contenedor cachea el NXDOMAIN en el proceso y no espeja nada. Un `wget` desde adentro da falso OK.

Del código

- El texto que el modelo escribe **junto a una tool** se perdía: el precio del domicilio se respondía y el cliente nunca lo recibía.
- `precio: 0` es **gratis**; `null` es pendiente. En los datos de Tu Bodega el 0 se usó como «no sé», así que migrar sin convertir regalaría mercancía.
- El modelo escribe **`**negrita**`** de Markdown, que en WhatsApp llega con los asteriscos a la vista.
- El **costo real está en el caché**: el prompt va con `cache_control`, así que los tokens cacheados no aparecen en `input_tokens`. Un turno reporta 333 y en realidad mueve ~15.100 por llamada al modelo.
- Con dos profesionales, un horario solo se ocupa cuando **ambos** están tomados. Tratarlo como ocupado con uno hacía decir «no hay cupo» con media agenda libre.
- El prompt **debe traer la fecha de hoy** o el modelo no resuelve «mañana a las 3». Va la fecha y no la hora: un timestamp al minuto rompería el caché en cada turno.

De la infraestructura

- El contenedor de Evolution está en dos redes, y el Chatsuite también tiene servicios llamados `postgres` y `redis`: con esos nombres, Evolution resolvía **al postgres del cliente**. Por eso se llaman `evo-postgres` y `evo-redis`.
- Si `docker compose up -d` falla a mitad, el contenedor queda creado **sin red** y un `up -d` posterior solo hace `start`: arranca sin red y espera para siempre.
- `pm2 restart --update-env` toma el entorno de quien lo invoca: sin pasarle la ruta del perfil, un bot reiniciaba con la configuración de **otro cliente**.
- Reescribir el sitio de nginx para sumar un `location` **borraría el bloque TLS** que puso certbot.